

Cours : 4 Ingénieurs Réseaux

Virtualisation des réseaux et des systèmes

TP2 : QEMU/KVM

Prof. BOUKLI-HACENE Sofiane

boukli@univ-sba.dz

EEDIS Laboratory

**Département d'informatique
Faculté des sciences exactes
Université Djilali Liabes**

18 octobre 2025



Lab Overview

- 1 Introduction to QEMU/KVM
- 2 Creating Virtual Machines
- 3 Managing Virtual Machines
- 4 Virtual Networking
- 5 Storage Management
- 6 Advanced Features
- 7 Introduction to QEMU Architecture Support
 - Installation and Setup
 - x86 Architecture
- 8 Lab Exercises
 - ARM Architecture
 - MIPS Architecture
 - RISC-V Architecture
 - Cross-Architecture Operations
- 9 Useful Resources and References
 - Hands-on Lab Exercises !
 - Practical Lab Exercises on Cross-Architecture

1 Introduction to QEMU/KVM

What is QEMU/KVM ?

QEMU (Quick EMUlator)

- Machine emulator and virtualizer
- Supports multiple architectures
- Can run without KVM (slower)
- Handles I/O device emulation

KVM (Kernel-based Virtual Machine)

- Linux kernel module
- Provides hardware virtualization
- Requires CPU virtualization extensions
- Intel VT-x or AMD-V

Together : QEMU provides emulation, KVM provides acceleration

Prerequisites and System Check

Check Virtualization Support

```
1 #!/bin/bash
2 # Check CPU virtualization support
3 egrep -c '(vmx|svm)' /proc/cpuinfo
4
5 # Check KVM module
6 lsmod | grep kvm
7
8 # Check if /dev/kvm exists
9 ls -la /dev/kvm
```

code/vm/check_virtualization.sh.txt

Install packages

Install QEMU/KVM

```
1 #!/bin/bash
2 # Ubuntu/Debian
3 sudo apt update
4 sudo apt install qemu-kvm qemu-utils libvirt-daemon-system
   libvirt-clients bridge-utils
5
6 # CentOS/RHEL
7 sudo yum install qemu-kvm qemu-img libvirt libvirt-python
   libvirt-client virt-install virt-viewer
8
9 # Start services
10 sudo systemctl enable libvirtd
11 sudo systemctl start libvirtd
12
13 # Add user to libvirt group
14 sudo usermod -a -G libvirt $USER
```

code/vm/install_qemu.sh.txt

2 Creating Virtual Machines

Creating Virtual Disk Images

Common Disk Formats

- **qcow2** : Copy-on-write, compression, snapshots
- **raw** : Direct access, simple format
- **vmdk** : VMware compatibility
- **vdi** : VirtualBox compatibility

Creating Virtual Disk Images

Creating Disk Images

```
1 #!/bin/bash
2 # Create qcow2 disk (dynamic allocation)
3 qemu-img create -f qcow2 ubuntu-server.qcow2 20G
4
5 # Create raw disk (pre-allocated)
6 qemu-img create -f raw windows10.raw 40G
7
8 # Create disk with backing file (for snapshots)
9 qemu-img create -f qcow2 -b ubuntu-server.qcow2 snapshot1.
   qcow2
10
11 # Convert between formats
12 qemu-img convert -f qcow2 -O vmdk ubuntu-server.qcow2 ubuntu
   -server.vmdk
```

code/vm/create_disks.sh.txt

Basic VM Creation and Installation

Explanation of Parameters

- `-m 2048` : 2GB RAM
- `-smp 2` : 2 CPU cores
- `-drive` : Storage configuration
- `-cdrom` : Installation media
- `-boot` : Boot order

Install from ISO

```
1 #!/bin/bash
2 # Basic VM installation
3 qemu-system-x86_64 \
4   -enable-kvm \
5   -name "My-VM" \
6   -m 2048 \
7   -smp 2 \
8   -drive file=ubuntu-server.qcow2,format=qcow2 \
9   -cdrom ubuntu-20.04.iso \
10  -boot order=dc \
11  -netdev user,id=net0 \
12  -device virtio-net-pci,netdev=net0
```

code/vm/install_vm.sh.txt

Advanced VM Configuration

Optimized VM with Multiple Disks

```
1 #!/bin/bash
2 # Advanced VM configuration with multiple disks and
   optimized settings
3 qemu -system-x86_64 \
4     -enable-kvm \
5     -name "Advanced-VM" \
6     -m 4096 \
7     -smp 4 \
8     -drive file=system.qcow2,if=virtio,format=qcow2,cache=
   writeback \
9     -drive file=data.qcow2,if=virtio,format=qcow2,cache=none \
10    -netdev user,id=net0,hostfwd=tcp::2222-:22 \
11    -device virtio-net-pci,netdev=net0 \
12    -boot order=c
```

code/vm/advanced_vm.sh.txt

3 Managing Virtual Machines

VM Lifecycle Management

Starting and Stopping VMs

```
1 #!/bin/bash
2 # VM lifecycle management commands
3
4 # Start VM in background
5 qemu-system-x86_64 -enable-kvm -m 2048 -drive file=vm.qcow2,
   format=qcow2 -daemonize
6
7 # Stop VM gracefully (requires acpi in VM)
8 echo "system_powerdown" | nc localhost 4444
9
10 # Force stop VM
11 pkill qemu-system-x86_64
12
13 # List running QEMU processes
14 ps aux | grep qemu-system
15
16 # Monitor VM status
17 virsh list --all
```

code/vm/vm_lifecycle.sh.txt

VM Lifecycle Management

Background Operation

```
1 #!/bin/bash
2 # Running VM in background with monitor interface
3 qemu-system-x86_64 \
4   -enable-kvm \
5   -m 2048 \
6   -drive file=vm.qcow2,format=qcow2 \
7   -monitor telnet:127.0.0.1:4444,server,nowait \
8   -daemonize
9
10 # Connect to monitor
11 telnet 127.0.0.1 4444
12
13 # Monitor commands examples:
14 # info status
15 # info block
16 # info network
17 # savevm snapshot1
18 # loadvm snapshot1
```

code/vm/background_vm.sh.txt

Monitoring and Performance

VM Monitoring Commands

```
1 #!/bin/bash
2 # VM monitoring and management commands
3
4 # Monitor VM performance
5 virsh dominfo vm-name
6 virsh nodeinfo
7 virt-top
8
9 # Check disk usage
10 qemu-img info vm-disk.qcow2
11
12 # Monitor network
13 virsh net-list
14 brctl show
15
16 # Check VM configuration
17 virsh dumpxml vm-name
18
19 # Resource usage
20 cat /proc/meminfo | grep -i hugepages
```

Snapshot Management

Snapshot Operations

- Create before major changes
- Use for backup points
- Test software installations
- Rollback failed updates

Snapshot Management

Creating and Managing Snapshots

```

1 #!/bin/bash
2 # Snapshot management commands
3
4 # Create internal snapshot (qcow2 only)
5 qemu-img snapshot -c boot-complete vm.qcow2
6
7 # List snapshots
8 qemu-img snapshot -l vm.qcow2
9
10 # Revert to snapshot
11 qemu-img snapshot -a boot-complete vm.qcow2
12
13 # Delete snapshot
14 qemu-img snapshot -d boot-complete vm.qcow2
15
16 # Create external snapshot
17 qemu-img create -f qcow2 -b vm.qcow2 snapshot1.qcow2
18
19 # Commit snapshot changes back to base
20 qemu-img commit snapshot1.qcow2
    
```

4 Virtual Networking

Network Configuration Types

User Mode Networking (NAT)

- Default network mode
- VM can access external network
- External cannot access VM
- Good for basic internet access

Bridged Networking

- VM appears as separate machine
- Gets IP from network DHCP
- Full network accessibility
- Requires host configuration

Network Configuration Examples

User Mode (NAT) Networking

```
1 #!/bin/bash
2 # User mode (NAT) networking examples
3
4 # Basic user networking
5 qemu-system-x86_64 \
6     -enable-kvm \
7     -m 1024 \
8     -drive file=vm.qcow2,format=qcow2 \
9     -netdev user,id=net0 \
10    -device e1000,netdev=net0
```

code/vm/user_network.sh.txt

Network Configuration Examples

User Mode (NAT) Networking

```
1 #!/bin/bash
2 # User networking with port forwarding
3
4 qemu-system-x86_64 \
5   -enable-kvm \
6   -m 1024 \
7   -drive file=vm.qcow2,format=qcow2 \
8   -netdev user,id=net0,hostfwd=tcp::2222-:22,hostfwd=tcp
9   ::8080-:80 \
10  -device virtio-net-pci,netdev=net0
```

code/vm/user_network1.sh.txt

Network Configuration Examples

User Mode (NAT) Networking

```
1 #!/bin/bash
2 # With DHCP and DNS
3
4 qemu-system-x86_64 \
5   -enable-kvm \
6   -m 1024 \
7   -drive file=vm.qcow2,format=qcow2 \
8   -netdev user,id=net0,net=192.168.100.0/24,dhcpstart
9   =192.168.100.50 \
   -device virtio-net-pci,netdev=net0
```

code/vm/user_network2.sh.txt

Network Configuration Examples

Bridged Networking

```
1 #!/bin/bash
2 # Bridged networking setup
3
4 # Create bridge on host
5 sudo ip link add name br0 type bridge
6 sudo ip link set br0 up
7 sudo ip addr add 192.168.1.100/24 dev br0
8
9 # Add physical interface to bridge
10 sudo ip link set eth0 master br0
11
12 # VM with bridged networking
13 qemu -system-x86_64 \
14     -enable-kvm \
15     -m 2048 \
16     -drive file=vm.qcow2,format=qcow2 \
17     -netdev bridge,id=net0,br=br0 \
18     -device virtio-net-pci,netdev=net0,mac=52:54:00:12:34:56
```

code/vm/bridged_network.sh.txt

Advanced Networking

Multiple Network Interfaces

```
1 #!/bin/bash
2 # Multiple network interfaces
3
4 qemu-system-x86_64 \
5   -enable-kvm \
6   -m 2048 \
7   -drive file=vm.qcow2,format=qcow2 \
8   \
9   # Management network (NAT)
10  -netdev user,id=mgmt,hostfwd=tcp::2222-:22 \
11  -device virtio-net-pci,netdev=mgmt,mac=52:54:00:12:34:01 \
12  \
13  # Data network (bridge)
14  -netdev bridge,id=data,br=br0 \
15  -device virtio-net-pci,netdev=data,mac=52:54:00:12:34:02
```

code/vm/multi_network.sh.txt

5 Storage Management

Advanced Storage Operations

Resizing Disk Images

```
1 #!/bin/bash
2 # Disk resizing operations
3
4 # Resize qcow2 disk (increase by 10GB)
5 qemu-img resize vm.qcow2 +10G
6
7 # Resize to specific size
8 qemu-img resize vm.qcow2 30G
9
10 # Resize raw disk
11 qemu-img resize vm.raw 50G
12
13 # Check disk information after resize
14 qemu-img info vm.qcow2
15
16 # Note: After resizing the image, you need to resize the
17 #       filesystem inside the VM
18 # For Linux: Use resize2fs (ext4)
19 # For Windows: Use Disk Management tool
```

code/vm/resize_disks.sh.txt

Storage Backends and Options

Different Storage Types

```

1 #!/bin/bash
2 # Different storage backends and options
3 # Local file storage
4 qemu-system-x86_64 \
5     -drive file=vm.qcow2,if=virtio,cache=writeback,discard=
6     ignore
7
8 # Direct block device
9 qemu-system-x86_64 \
10    -drive file=/dev/sdb,if=virtio,cache=none
11
12 # NFS storage
13 qemu-system-x86_64 \
14    -drive file=nfs://192.168.1.100/exports/disk.qcow2,if=
15    virtio
16
17 # LVM storage
18 qemu-system-x86_64 \
19    -drive file=/dev/vg0/vm-disk,if=virtio,cache=none
  
```

code/vm/storage_backends.sh.txt

Storage Backends and Options

Storage Performance Tips

- Use `cache=none` for database VMs
- Pre-allocate disk space
- Use SSD storage
- Consider raw format for performance

Security Considerations

- Encrypt sensitive VM disks
- Use Linux Unified Key Setup (LUKS) encryption
- Secure snapshot storage
- Regular backups

6 Advanced Features

Live Migration

Setting up Shared Storage

```
1 #!/bin/bash
2 # Setting up shared storage for live migration
3
4 # Using NFS for shared storage
5 sudo apt install nfs-kernel-server
6
7 # Configure NFS export on storage server
8 echo "/var/lib/libvirt/images 192.168.1.0/24(rw, sync,
   no_subtree_check)" >> /etc/exports
9 sudo systemctl restart nfs-kernel-server
10
11 # Mount NFS on compute nodes
12 mkdir -p /var/lib/libvirt/images
13 echo "192.168.1.100:/var/lib/libvirt/images /var/lib/libvirt
   /images nfs defaults 0 0" >> /etc/fstab
14 mount -a
```

code/vm/shared_storage.sh.txt

Live Migration

Live Migration Process

```

1  #!/bin/bash
2  # Live migration process
3  # Pre-migration checks
4  virsh migrate--setspeed vm-name --migrate-speed 1000
5  virsh migrate--getspeed vm-name
6
7  # Live migration to another host
8  virsh migrate --live vm-name qemu+ssh://dest-host/system
9
10 # Migration with specific parameters
11 virsh migrate --live vm-name \
12     qemu+ssh://dest-host/system \
13     --unsafe \
14     --verbose \
15     --timeout 120 \
16     --migrateuri tcp://dest-host:49152
    
```

code/vm/live_migration.sh.txt

7 Introduction to QEMU Architecture Support

QEMU's Multi-Architecture Capabilities

Supported Architectures

QEMU can emulate a wide range of CPU architectures :

- **x86 (32/64-bit)** : Intel/AMD processors
- **ARM (32/64-bit)** : ARMv7, ARMv8, Cortex series
- **MIPS (32/64-bit)** : MIPS I-V, microMIPS
- **RISC-V (32/64/128-bit)** : RISC-V RV32/64/128
- **PowerPC** : PowerPC 32/64-bit
- **SPARC** : SPARC v8/v9
- **s390x** : IBM System z

Key Concept

QEMU uses **dynamic binary translation** to convert guest CPU instructions to host CPU instructions in real-time.

QEMU System vs User Mode Emulation

System Mode Emulation

- Full system virtualization
- Emulates CPU, memory, devices
- Boots complete OS
- Used for : VMs, firmware development
- Command : `qemu-system-*`

User Mode Emulation

- Runs single programs
- No device emulation
- Cross-compilation testing
- Used for : development, testing
- Command : `qemu-*`

Example

`qemu-system-x86_64` VS `qemu-arm`

7 Introduction to QEMU Architecture Support

■ Installation and Setup

- x86 Architecture
- ARM Architecture
- MIPS Architecture
- RISC-V Architecture
- Cross-Architecture Operations

Installing QEMU with Multi-Architecture Support

Ubuntu/Debian Installation

```
1 #!/bin/bash
2 # Install QEMU with multi-architecture support on Ubuntu/
   Debian
3
4 sudo apt update
5 sudo apt install qemu-system-x86 qemu-system-arm qemu-system-
   -mips qemu-system-ppc qemu-system-sparc qemu-system-
   riscv64 qemu-utils qemu-user qemu-user-static
6
7 # Additional architecture support
8 sudo apt install qemu-efi-aarch64 qemu-efi-arm
9
10 # Development tools
11 sudo apt install gcc-aarch64-linux-gnu gcc-mips-linux-gnu
   gcc-riscv64-linux-gnu
```

code/vm/install_multiarch.sh.txt

Verifying Architecture Support

Check Available QEMU Binaries

```
1 #!/bin/bash
2 # Check available QEMU system emulators
3
4 echo "=== Available QEMU System Emulators ==="
5 ls /usr/bin/qemu-system-*
6
7 echo -e "\n=== Checking specific architectures ==="
8 for arch in x86_64 aarch64 arm mips mips64 ppc riscv64; do
9     if command -v qemu-system-$arch &> /dev/null; then
10         echo "    qemu-system-$arch: $(which qemu-system-$arch)"
11     else
12         echo "    qemu-system-$arch: Not found"
13     fi
14 done
```

code/vm/check_arch_support.sh.txt

Verifying Architecture Support

QEMU Version and Capabilities

```
1 #!/bin/bash
2 # Check QEMU capabilities and version
3
4 echo "=== QEMU Version ==="
5 qemu-system-x86_64 --version
6
7 echo -e "\n=== Available Machines ==="
8 qemu-system-x86_64 -machine help | head -20
9
10 echo -e "\n=== Available CPUs ==="
11 qemu-system-x86_64 -cpu help | head -10
12
13 echo -e "\n=== Acceleration Options ==="
14 qemu-system-x86_64 -accel help
```

code/vm/check_capabilities.sh.txt

7 Introduction to QEMU Architecture Support

■ Installation and Setup

■ x86 Architecture

■ ARM Architecture

■ MIPS Architecture

■ RISC-V Architecture

■ Cross-Architecture Operations

x86 Architecture Overview

x86 Features in QEMU

- Most mature and optimized
- Hardware acceleration via KVM
- Supports Intel VT-x and AMD-V
- Multiple CPU models : qemu64, Nehalem, Haswell, etc.
- SMP support up to 255 CPUs
- PCI/PCIe device emulation

Creating x86 Virtual Machines

Basic x86_64 VM

```
1 #!/bin/bash
2 # Basic x86_64 virtual machine
3
4 qemu-system-x86_64 \
5     -machine pc,accel=kvm \
6     -cpu host \
7     -smp 4 \
8     -m 4G \
9     -drive file=x86-vm.qcow2,format=qcow2 \
10    -netdev user,id=net0 \
11    -device e1000,netdev=net0 \
12    -boot order=cd \
13    -cdrom ubuntu-22.04-desktop-amd64.iso
```

code/vm/x86_basic_vm.sh.txt

Creating x86 Virtual Machines

Advanced x86 VM with Specific CPU

```
1 #!/bin/bash
2 # Advanced x86 VM with specific CPU features
3
4 qemu-system-x86_64 \
5     -machine q35,accel=kvm \
6     -cpu Nehalem,+aes,+avx \
7     -smp 8,cores=4,threads=2,sockets=1 \
8     -m 8G \
9     -drive file=win11.qcow2,if=virtio,format=qcow2 \
10    -drive file=driver.iso,media=cdrom \
11    -netdev user,id=net0,hostfwd=tcp::2222-:22 \
12    -device virtio-net-pci,netdev=net0 \
13    -vga virtio \
14    -usb -device usb-tablet \
15    -rtc base=localtime,clock=host \
16    -boot menu=on
```

code/vm/x86_advanced_vm.sh.txt

x86 CPU Models and Features

Available CPU Models

```
1 #!/bin/bash
2 # List and test x86 CPU models
3
4 echo "=== Available x86 CPU Models ==="
5 qemu-system-x86_64 -cpu help | grep x86
6
7 echo -e "\n=== Testing Specific CPU ==="
8 qemu-system-x86_64 -cpu Cascadelake-Server -machine pc,accel
   =tcg -nographic -nic none -m 512 -kernel /dev/null 2>&1
   | grep "CPU model"
9
10 echo -e "\n=== CPU Feature Flags ==="
11 qemu-system-x86_64 -cpu qemu64,enforce -machine none -
   nographic -nic none <<EOF
12 quit
13 EOF
```

code/vm/x86_cpu_models.sh.txt

7 Introduction to QEMU Architecture Support

- Installation and Setup
- x86 Architecture
- **ARM Architecture**
- MIPS Architecture
- RISC-V Architecture
- Cross-Architecture Operations

ARM Architecture Support in QEMU

ARM Emulation Features

- ARMv7-A (32-bit) and ARMv8-A (64-bit)
- Various Cortex-A series CPUs
- Versatile Express and Virt machine types
- GIC (Generic Interrupt Controller)
- TrustZone emulation
- NEON SIMD support

Creating ARM Virtual Machines

ARM 64-bit (aarch64) VM

```
1 #!/bin/bash
2 # ARM 64-bit virtual machine
3
4 qemu-system-aarch64 \
5     -machine virt \
6     -cpu cortex-a72 \
7     -smp 4 \
8     -m 4G \
9     -drive file=arm64-vm.qcow2,format=qcow2,if=virtio \
10    -netdev user,id=net0,hostfwd=tcp::2222-:22 \
11    -device virtio-net-pci,netdev=net0 \
12    -device virtio-gpu-pci \
13    -device usb-kbd \
14    -device usb-mouse \
15    -bios /usr/share/qemu-efi-aarch64/QEMU_EFI.fd
```

code/vm/arm64_vm.sh.txt

ARM 32-bit VM

ARM Machine Types and Devices

Available ARM Machines

```
1 #!/bin/bash
2 # List ARM machine types
3
4 echo "=== ARM Machine Types ==="
5 qemu-system-aarch64 -machine help | grep arm
6 qemu-system-arm -machine help | grep arm
7
8 echo -e "\n=== Common ARM Machines ==="
9 echo "virt: ARM Virtual Machine (recommended)"
10 echo "vexpress-a9: ARM Versatile Express"
11 echo "raspi2: Raspberry Pi 2"
12 echo "raspi3: Raspberry Pi 3"
13
14 # Test specific machine
15 qemu-system-aarch64 -machine virt,dumpdtb=virt.dtb
```

code/vm/arm_machines.sh.txt

ARM Specific Devices

ARM UEFI and Boot Process

ARM with UEFI Boot

```
1 #!/bin/bash
2 # ARM with UEFI boot
3
4 # Check for UEFI firmware
5 if [ ! -f "/usr/share/qemu-efi-aarch64/QEMU_EFI.fd" ]; then
6     echo "Installing ARM UEFI firmware..."
7     sudo apt install qemu-efi-aarch64
8 fi
9
10 qemu-system-aarch64 \
11     -machine virt \
12     -cpu cortex-a72 \
13     -smp 4 \
14     -m 4G \
15     -drive file=arm64-uefi.qcow2,format=qcow2,if=virtio \
16     -drive if=pflash,format=raw,file=/usr/share/qemu-efi-
17     aarch64/QEMU_EFI.fd \
18     -netdev user,id=net0 \
19     -device virtio-net-pci,netdev=net0 \
20     -boot menu=on
```

7 Introduction to QEMU Architecture Support

- Installation and Setup
- x86 Architecture
- ARM Architecture
- **MIPS Architecture**
- RISC-V Architecture
- Cross-Architecture Operations

Creating MIPS Virtual Machines

MIPS 64-bit VM

```

1 #!/bin/bash
2 # MIPS 64-bit virtual machine
3
4 qemu-system-mips64 \
5     -machine malta \
6     -cpu 5KEf \
7     -smp 2 \
8     -m 2G \
9     -drive file=mips64-vm.qcow2,format=qcow2 \
10    -netdev user,id=net0 \
11    -device e1000,netdev=net0 \
12    -kernel vmlinux-5.4.0-4-5kc-malta \
13    -initrd initrd.img-5.4.0-4-5kc-malta \
14    -append "root=/dev/sda1 console=ttyS0" \
15    -nographic
    
```

code/vm/mips64_vm.sh.txt

MIPS 32-bit VM



MIPS Machine Configuration

MIPS Machine Types

```

1 #!/bin/bash
2 # MIPS machine types
3
4 echo "=== MIPS Machine Types ==="
5 qemu-system-mips64 -machine help | grep mips
6 qemu-system-mips -machine help | grep mips
7
8 echo -e "\n=== Common MIPS Machines ==="
9 echo "malta: MIPS Malta board (most common)"
10 echo "magnum: MIPS Magnum board"
11 echo "mipssim: MIPS MIPSSim board"
12
13 # Test Malta machine
14 qemu-system-mips -machine malta -cpu 24Kf -nographic -nic
    none -m 128 -kernel /dev/null
    
```

code/vm/mips_machines.sh.txt

MIPS Machine Configuration

MIPS CPU Models

```

1 #!/bin/bash
2 # MIPS CPU models
3
4 echo "=== MIPS CPU Models ==="
5 qemu-system-mips64 -cpu help | grep MIPS
6 qemu-system-mips -cpu help | grep MIPS
7
8 echo -e "\n=== Popular MIPS CPUs ==="
9 echo "24Kf: MIPS32 24K with FPU"
10 echo "5KEf: MIPS64 5KE with FPU"
11 echo "P5600: MIPS32 P5600"
12 echo "I6400: MIPS64 I6400"
13
14 # Test different CPUs
15 for cpu in 24Kf 5KEf P5600; do
16     echo "Testing CPU: $cpu"
17     timeout 5s qemu-system-mips -machine malta -cpu $cpu -
18     nographic -nic none -m 64 -kernel /dev/null
19 done
    
```

code/vm/mips_cpus.sh.txt

7 Introduction to QEMU Architecture Support

- Installation and Setup
- x86 Architecture
- ARM Architecture
- MIPS Architecture
- **RISC-V Architecture**
- Cross-Architecture Operations

RISC-V Architecture Support

RISC-V Features

- RV32, RV64, RV128 support
- Multiple privilege levels
- Various extensions (M, A, F, D, C)
- Virt machine for general purpose
- sifive_u for SiFive U-series
- OpenTitan for security research

Creating RISC-V Virtual Machines

RISC-V 64-bit VM

```
1 #!/bin/bash
2 # RISC-V 64-bit virtual machine
3
4 qemu-system-riscv64 \
5     -machine virt \
6     -cpu rv64 \
7     -smp 4 \
8     -m 4G \
9     -drive file=riscv64-vm.qcow2,format=qcow2,if=virtio \
10    -netdev user,id=net0,hostfwd=tcp::2222-:22 \
11    -device virtio-net-device,netdev=net0 \
12    -device virtio-gpu-pci \
13    -device usb-kbd \
14    -device usb-mouse \
15    -bios /usr/lib/riscv64-linux-gnu/opensbi/generic/fw_jump
16    .elf \
17    -kernel /usr/lib/u-boot/qemu-riscv64_smode/u-boot.elf
```

code/vm/riscv64_vm.sh.txt

Creating RISC-V Virtual Machines

RISC-V with Specific Extensions

```
1 #!/bin/bash
2 # RISC-V with specific extensions
3
4 qemu-system-riscv64 \
5     -machine virt \
6     -cpu rv64,zba=true,zbb=true,zbc=true,zbs=true \
7     -smp 2 \
8     -m 2G \
9     -drive file=vm.qcow2,format=qcow2,if=virtio \
10    -netdev user,id=net0 \
11    -device virtio-net-device,netdev=net0
12
13 # Check available extensions
14 qemu-system-riscv64 -cpu help | grep rv64
```

code/vm/riscv_extensions.sh.txt

7 Introduction to QEMU Architecture Support

- Installation and Setup
- x86 Architecture
- ARM Architecture
- MIPS Architecture
- RISC-V Architecture
- Cross-Architecture Operations

Cross-Architecture Installation Methods

Using Architecture-Specific Installers

```
1 #!/bin/bash
2 # Cross-architecture installation methods
3
4 # Download architecture-specific installers
5 echo "=== Architecture-Specific Installers ==="
6 wget https://cdimage.debian.org/debian-cd/current/arm64/iso-
   cd/debian-11.2.0-arm64-netinst.iso
7 wget https://cdimage.debian.org/debian-cd/current/mips/iso-
   cd/debian-11.2.0-mips-netinst.iso
8 wget https://cdimage.debian.org/debian-cd/current/mipsel/iso-
   cd/debian-11.2.0-mipsel-netinst.iso
9
10 # Use cloud images for quick start
11 wget https://cloud-images.ubuntu.com/focal/current/focal-
   server-cloudimg-arm64.img
12 wget https://cloud-images.ubuntu.com/focal/current/focal-
   server-cloudimg-ppc64el.img
```

code/vm/cross_arch_install.sh.txt

Boot Methods Across Architectures

Architecture-Specific Boot Methods

```
1 #!/bin/bash
2 # Architecture-specific boot methods
3
4 echo "=== x86 Boot Methods ==="
5 echo "BIOS: -boot order=cd"
6 echo "UEFI: -bios /usr/share/ovmf/OVMF.fd"
7
8 echo -e "\n=== ARM Boot Methods ==="
9 echo "UEFI: -bios /usr/share/qemu-efi-aarch64/QEMU_EFI.fd"
10 echo "Direct Kernel: -kernel vmlinuz -initrd initrd.img"
11
12 echo -e "\n=== RISC-V Boot Methods ==="
13 echo "OpenSBI: -bios opensbi-riscv64-generic-fw_dynamic.bin"
14 echo "Direct: -kernel Image"
15
16 echo -e "\n=== MIPS Boot Methods ==="
17 echo "Direct Kernel: -kernel vmlinux -append 'console=ttyS0'
    "
```

code/vm/arch_boot_methods.sh.txt

8 Lab Exercises

8 Lab Exercises

- Hands-on Lab Exercises !
- Practical Lab Exercises on Cross-Architecture

Hands-on Lab Exercises

1 Exercise 1 : Basic VM Setup

- Create a Ubuntu VM with 2GB RAM, 20GB disk
- Install using bridged networking
- Take a snapshot after installation

2 Exercise 2 : Network Configuration

- Set up an isolated virtual network
- Create two VMs on the same network
- Test connectivity between VMs

3 Exercise 3 : Storage Management

- Add a second virtual disk to existing VM
- Resize the primary disk
- Convert disk format from qcow2 to raw

8 Lab Exercises

- Hands-on Lab Exercises !
- Practical Lab Exercises on Cross-Architecture

Lab Exercise 2 : Real-World Scenarios

5 Embedded System Emulation

- Emulate Raspberry Pi (ARM)
- Boot embedded Linux
- interface GPIO emulation
- Test embedded applications

6 Network Device Emulation

- Create MIPS-based router VM
- Install OpenWRT
- Configure network interfaces
- Test routing functionality

9 Useful Resources and References

Useful Resources and References

Documentation

- QEMU System Emulation Targets :
<https://www.qemu.org/docs/master/system/targets.html>
- QEMU ARM System Emulator :
<https://www.qemu.org/docs/master/system/target-arm.html>
- QEMU MIPS System Emulator :
<https://www.qemu.org/docs/master/system/target-mips.html>
- QEMU RISC-V System Emulator :
<https://www.qemu.org/docs/master/system/target-riscv.html>

Useful Resources and References

Pre-built Images

- Ubuntu Cloud Images : <https://cloud-images.ubuntu.com/>
- Debian Netinst Images : <https://www.debian.org/distrib/netinst>
- Arch Linux ARM : <https://archlinuxarm.org/>
- Fedora Architecture Variants : <https://fedoraproject.org/>

Questions !!

Contact :

boukli@univ-sba.dz